Processing Data in External Storage

Chapter 21

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Look at External Storage

- External storage
  - Used when program reads/writes data to/from a C++ file
- Generally there is more external storage than internal memory
- You can create files for either sequential access or direct access
- Many programming languages, including C++, support both sequential access and direct access of files.
- Direct access files essential for external data collections

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Look at External Storage

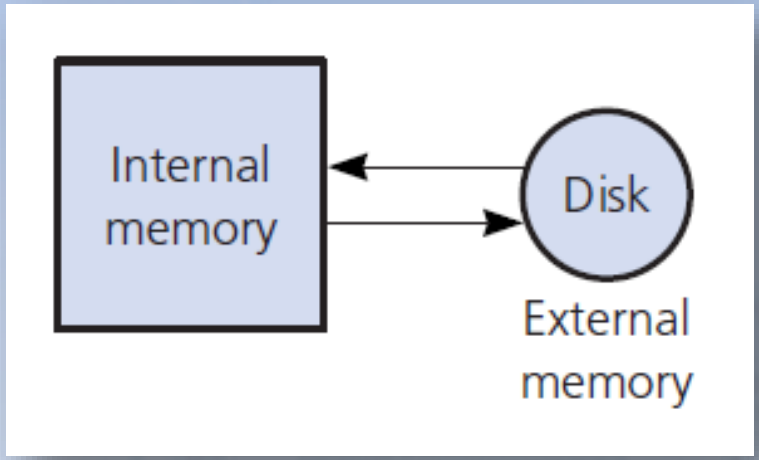
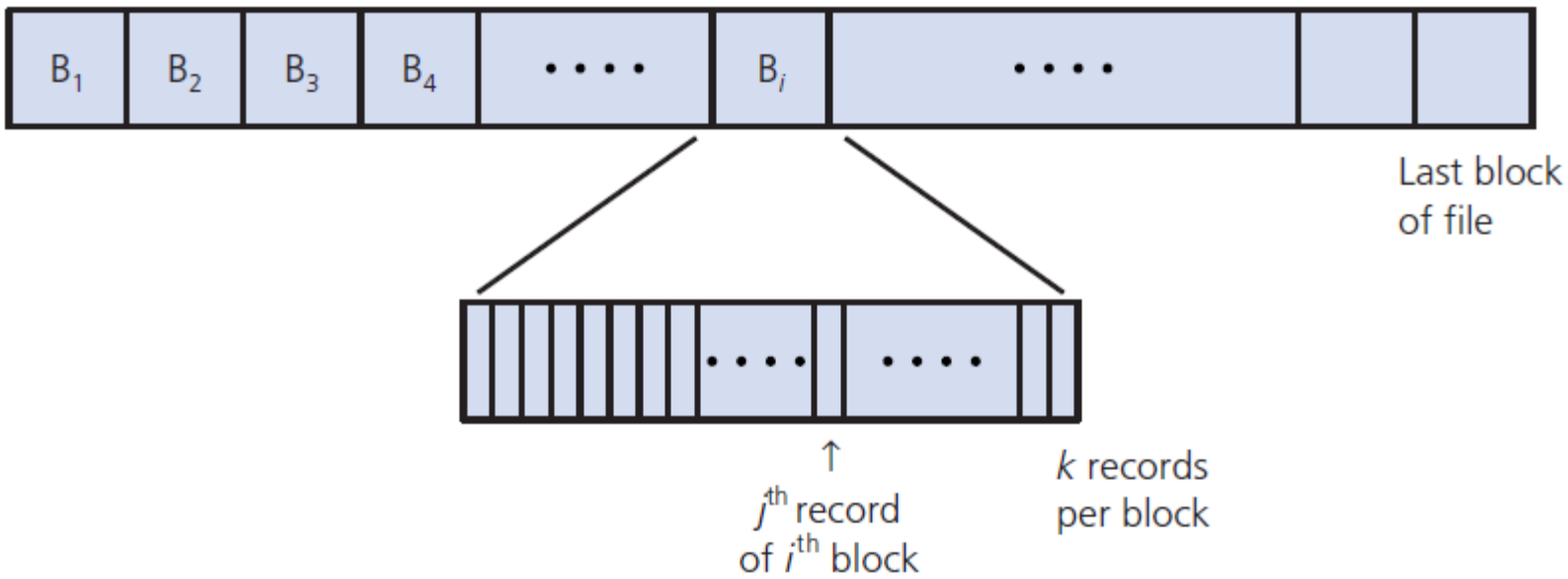


FIGURE 21-1 Internal and external memory

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Look at External Storage



- The size of a block—that is, the number of bits of data it can contain—is determined by both the hardware configuration and the system software of the computer.
- In general, an individual program has no control over this size. Therefore, the number of records in a block is a function of the size of the records in the file.

FIGURE 21-2 A file partitioned into blocks of records

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Look at External Storage

- Direct access I/O
  - Involves blocks instead of records
  - All I/O is at the block level rather than at the record level.
  - You can read and write a block of records, but you cannot read or write an individual record.
- Buffer stores data (blocks) temporarily
- Record updated within block (in buffer)
- Work to minimize block I/O
  - Takes more time for disk access

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorting Data in an External File

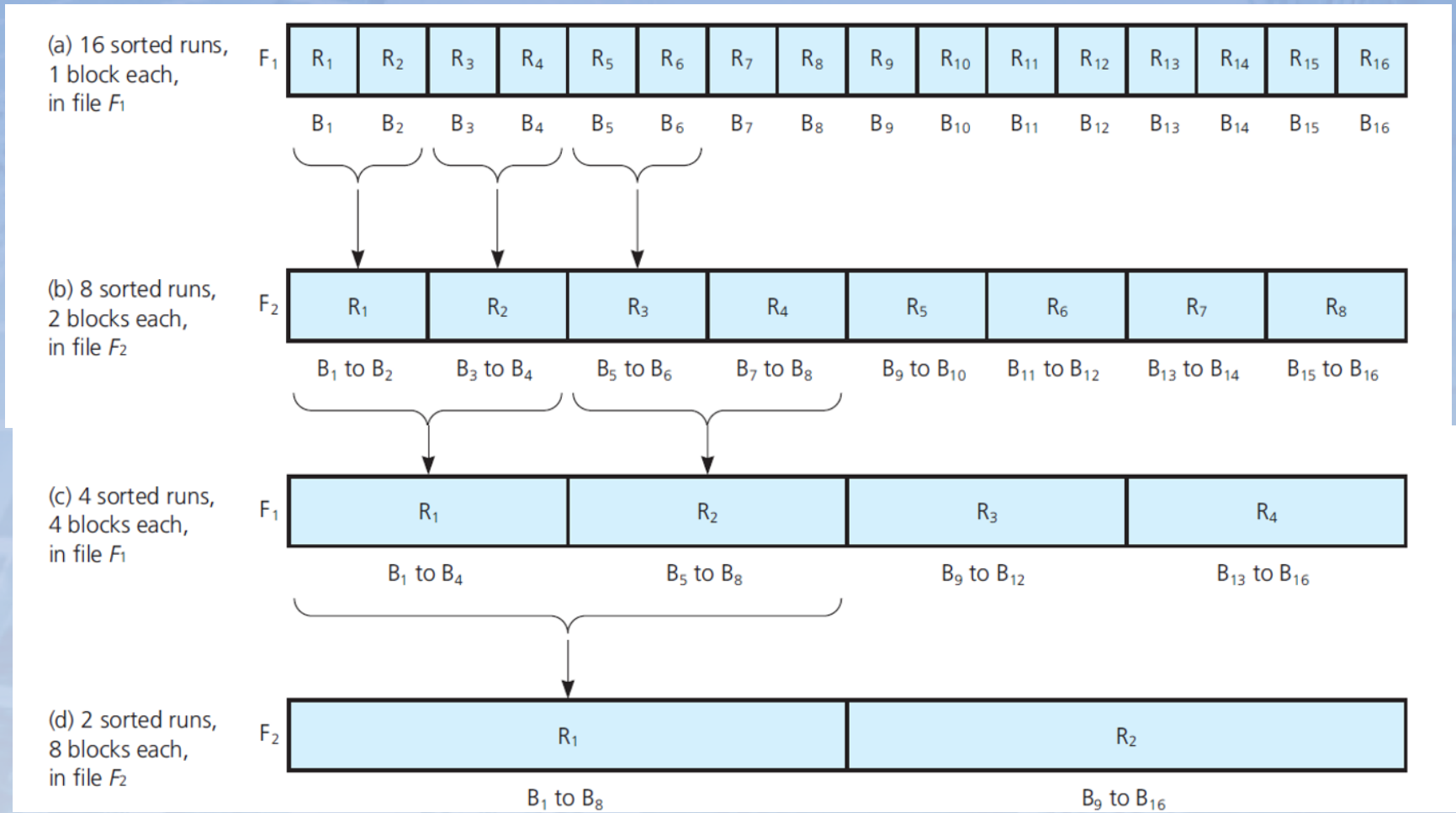
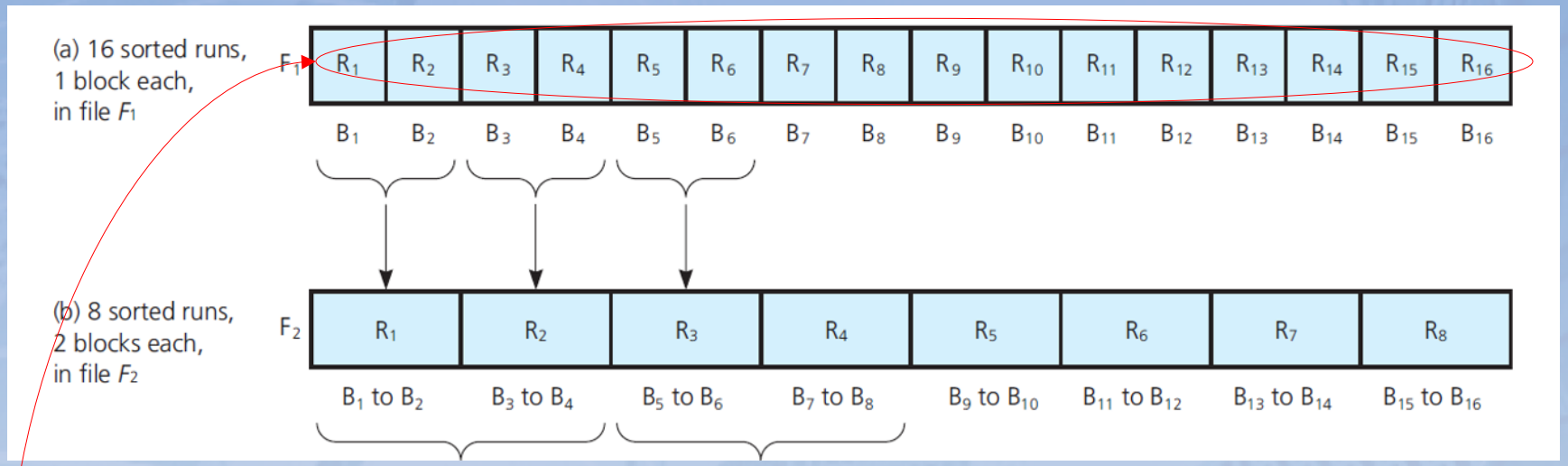


Figure 21-3 Sorting a block of an external file F by merging the results of internal sorts and using two external work files F_1 and F_2

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorting Data in an External File

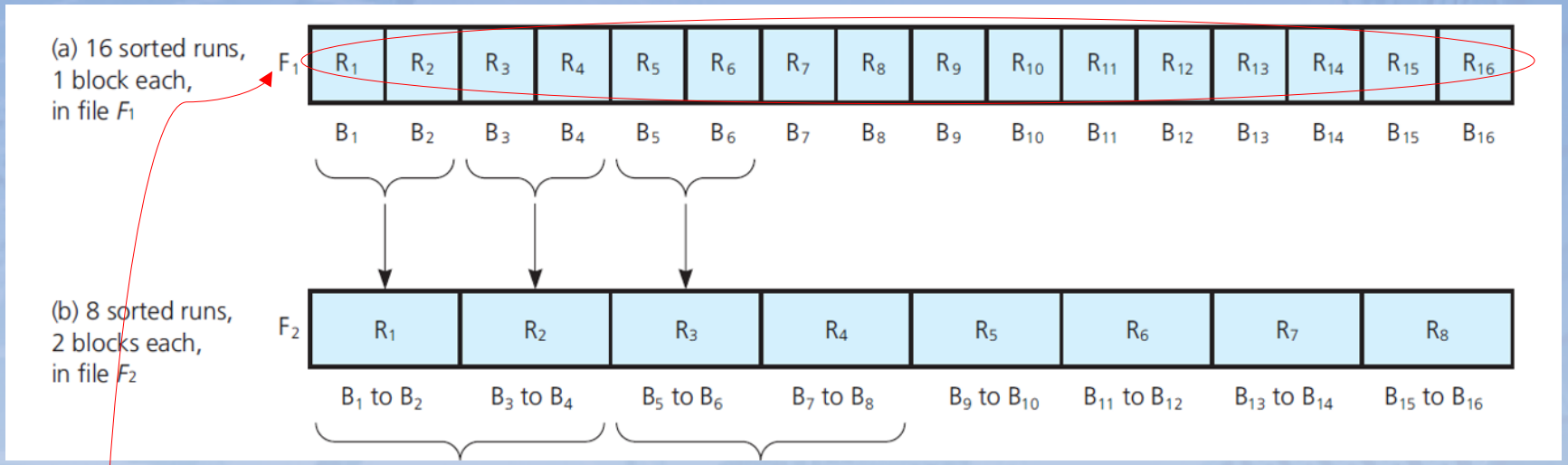


- An external file contains 1,600 employee records.
- You want to sort these records by Social Security number.
- Each block contains 100 records, and thus the file contains 16 blocks B1, B2, and so on to B16.
- Assume that the program can access only enough internal memory to manipulate about 300 records (three blocks' worth) at one time.
- The file is far too large to fit into internal memory all at once.
- Merge sort - merge two sorted segments—such as arrays—of data records into a third sorted segment that is the combination of the two.

Figure 21-3 Sorting a block of an external file F by merging the results of internal sorts and using two external work files F_1 and F_2

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorting Data in an External File

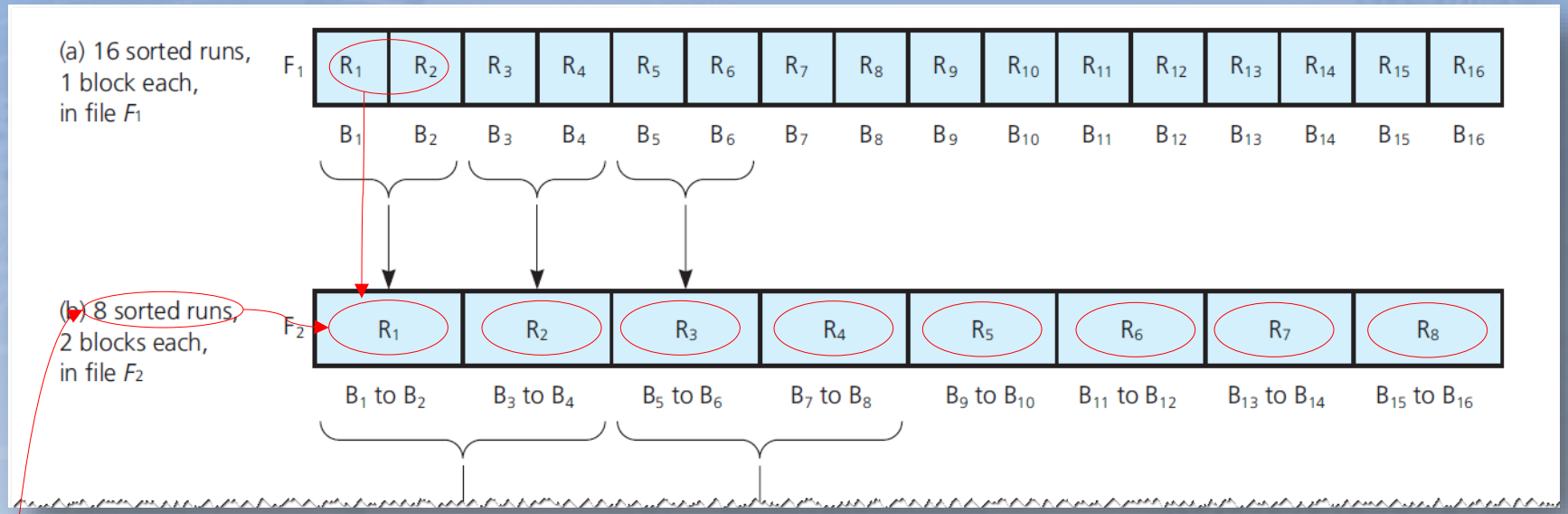


- Read a block from F into internal memory
- Sort its records by using an internal sort, and
- Write the sorted block out to F1 before you read the next block from F.
- After you process all 16 blocks of F, F1 contains 16 sorted runs R1, R2, and so on to R16
  - F1 contains 16 blocks of records, with the records within each block sorted among themselves, as Figure 21-3 a illustrates

Figure 21-3 Sorting a block of an external file F by merging the results of internal sorts and using two external work files F_1 and F_2

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorting Data in an External File

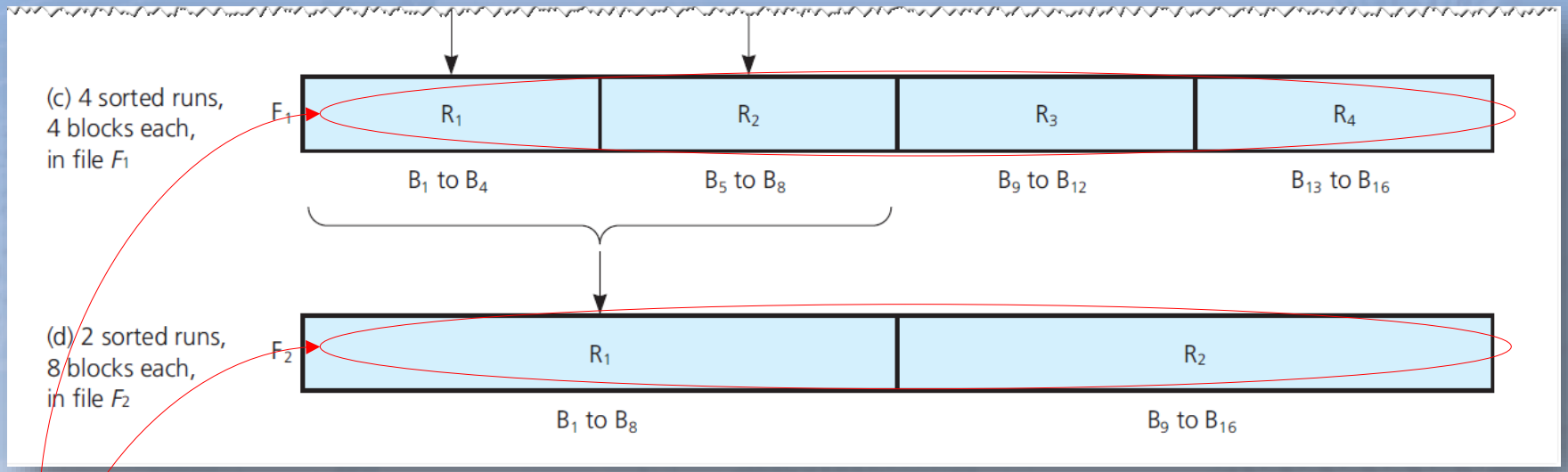


- Each merge step merges pairs of sorted runs to form larger sorted runs.
- With each merge step, the number of blocks in each sorted run doubles, and thus the total number of sorted runs is halved.
- Figure 21-3b shows the first merge step merges eight pairs of sorted runs from F1 (R1 with R2, R3 with R4, . . ., R15 with R16) to form eight sorted runs, each two blocks long, which are written to F2.

Figure 21-3 Sorting a block of an external file F by merging the results of internal sorts and using two external work files F_1 and F_2

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorting Data in an External File

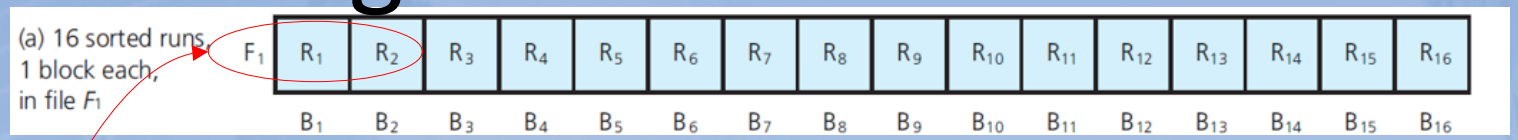


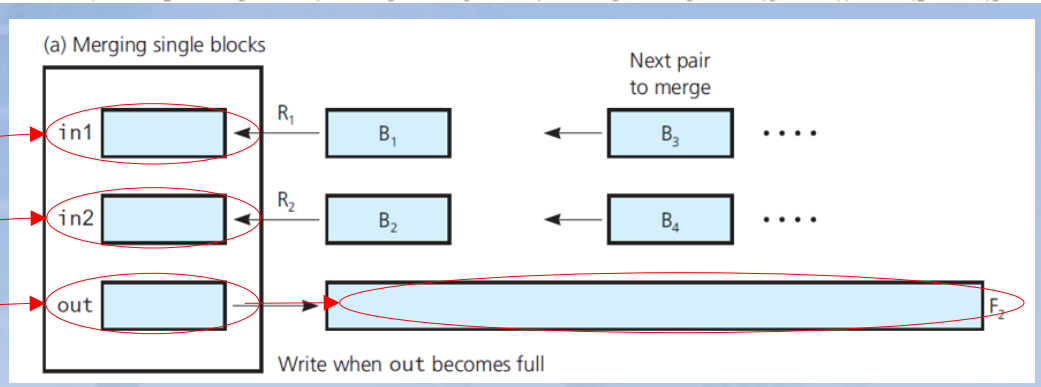
- The next merge step merges four pairs of sorted runs from F2 (R1 with R2, R3 with R4, . . ., R7 with R8) to form four sorted runs, each four blocks long, which are written back to F1, as Figure 21-3 c illustrates.
- The next step merges the two pairs of sorted runs from F1 to form two sorted runs, which are written to F2. (See Figure 21-3 d.)
- The final step merges the two sorted runs into one, which is written to F1. At this point, F1 will contain all of the records of the original file in sorted order

Figure 21-3 Sorting a block of an external file F by merging the results of internal sorts and using two external work files F_1 and F_2

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorting Data in an External File



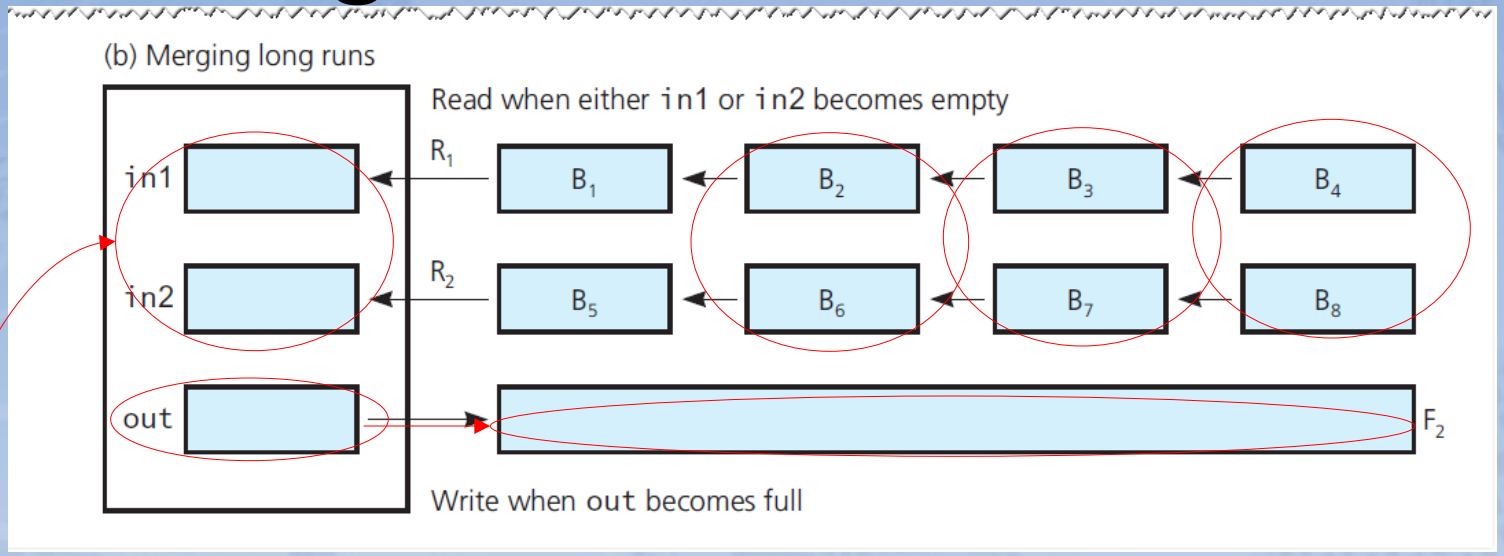


- **Merge the runs a piece at a time, due to internal memory limitation**
- Divide the program's internal memory into three arrays, in1, in2, and out, each capable of holding 100 records (the block size).
- Read block-sized pieces of the runs into the two in arrays and merge them into the out array.
- If an in array is exhausted, read the next piece of the run into an in array
- When the out array becomes full, write it to one of the files.
- To begin merge, start this step with the pair of runs R1 and R2, which are in the first and second blocks, respectively, of the file F1. (See Figure 21-3 a.)
- At first merge step each run contains only one block, an entire run can fit into one of the in arrays. You can thus read R1 and R2 into the arrays in1 and in2, and then merge in1 and in2 into out.
- However, although the result of merging in1 and in2 is a sorted run two blocks long (200 records), out can hold only one block (100 records). Thus, when in the course of the merge out becomes full, you write its contents to the first block of F2, as Figure 21-4 a illustrates

Figure 21-4 Phase 2 of an external sort: Merging sorted runs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorting Data in an External File

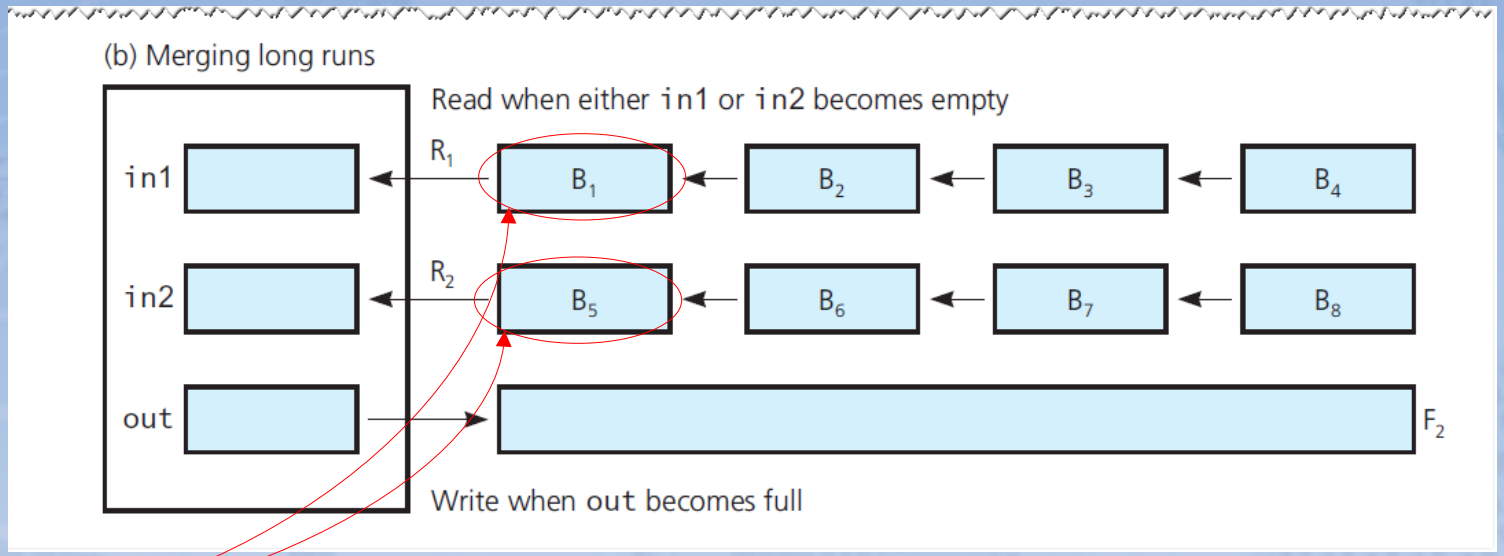


- The merging of in1 and in2 into out then resumes.
- The array out will become full for a second time only after all of the records in in1 and in2 are exhausted.
  - At that time, write the out contents to the second block of F2.
  - Merge the remaining seven pairs from F in the same manner and append the resulting runs to F2.
- This first merge step is easier than the others, because the initial runs are only one block in size, and thus each can fit entirely into one of the in arrays.

Figure 21-4 Phase 2 of an external sort: Merging sorted runs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Sorting Data in an External File (cont.)

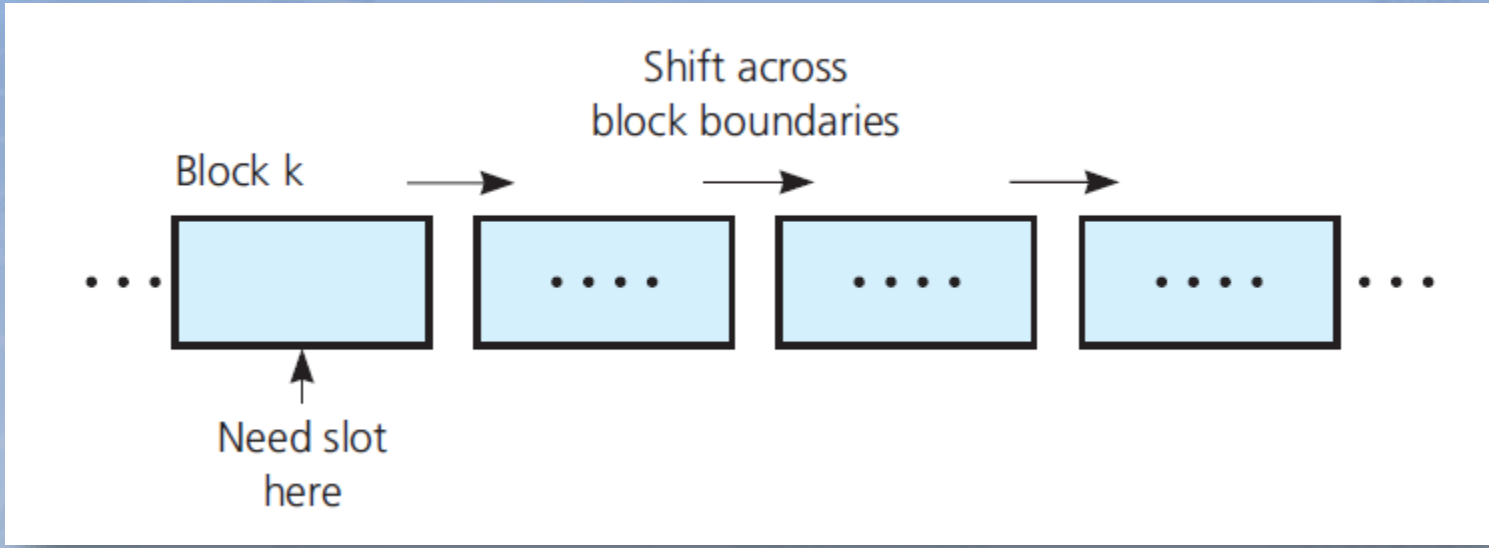


- In subsequent merge steps, read the first block of R1 —which is the first block B1 of the file—into in1, and read the first block of R2 —which is B5 —into in2, as Figure 21-4 b illustrates.
- Then, as it did earlier, the algorithm merges in1 and in2 into out.
- As soon as you finish moving all of the records from either in1 or in2, you must read the next block from the corresponding run; e.g., if you finish in2 first, you must read the next block of R2 —which is B6 —into in2 before the merge can continue.
- Detect when the in arrays become exhausted as well as when the out array becomes full.

Figure 21-4 Phase 2 of an external sort: Merging sorted runs

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Basic Data Management Operations on External Dictionaries

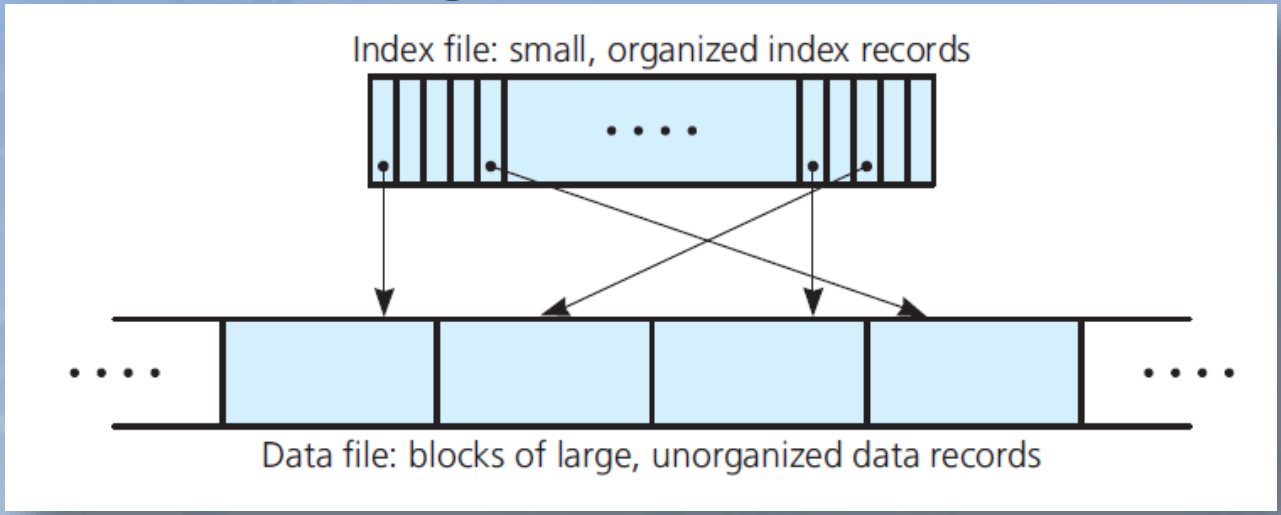


- One simple dictionary implementation - store the records in order by their search key, perhaps sorting the file by using the external merge sort algorithm developed in the previous section.
- Once sorted, you can easily traverse the file in sorted order.
- PRO - To perform the retrieval operation on the sorted file, you can use a binary search algorithm.
  - Number of records in the block is typically small, and thus the time required to scan the block sequentially is insignificant compared to the time required to read the block from the file; common simply to scan the block sequentially
- CON - add and remove operations must shift dictionary items; very expensive
  - Inserting into block k, requires shift in block k, and every block after it.

Figure 21-5 Shifting across block boundaries

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Indexing an External File



- Leave the data file in a disorganized state; maintain an organized index to it
- To locate a record in the data file, search the index for the corresponding entry
- An index to the data file is simply another file, called the index file, that contains an index record for each record in the data file, just as a library catalog contains an entry for each book in the library.
- An index record has two parts:
  1. A key, which contains the same value as the search key of its corresponding record in the data file, and
  2. A "pointer," which shows the number of the block in the data file that contains this data record. (This index record's pointer is an integer, not a C++ pointer.)
- You can determine which block of the data file contains the record whose search key equals searchKey by searching the index file for the index record whose key equals searchKey.

Figure 21-6 A data file with an index

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

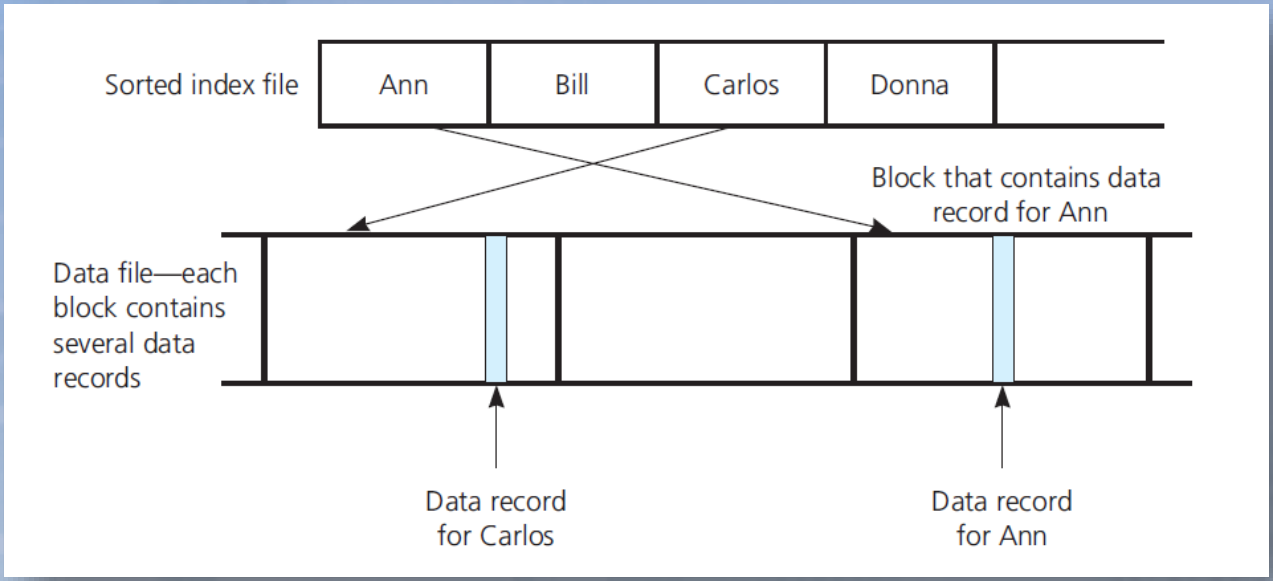
Indexing an External File

Advantages of an index file

- Index record smaller than a data record
- Data file need not be kept in any particular order
- Possible to maintain several indexes simultaneously; e.g., <name, pointer> and <socSec, pointer>
- Index file reduces number of block accesses
- Shift index records, not data records

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Indexing an External File

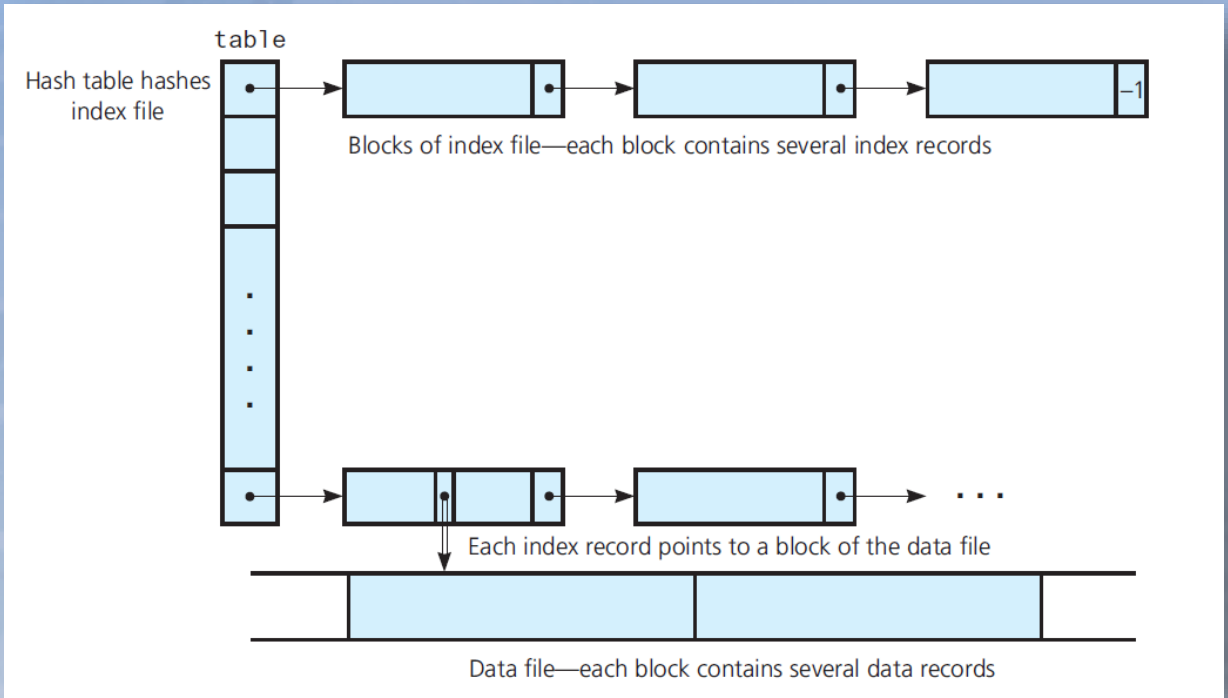


- Index records are far smaller than the data records, the index file contains far fewer blocks than the data file
- Performing insertion or removal, you shift only index records; you do not keep the data file in any particular order, so you can insert a new data record into any convenient location in the data file.
- You never need to shift records in the data file; shift records in the index file to create an opening for a corresponding index entry in its proper sorted position.
- The index file contains many fewer blocks than the data file, so the maximum number of block accesses required is greatly reduced
- This scheme is an improvement over maintaining a sorted data file
- Better implementations by using either hashing or search trees to organize the index file

Figure 21-7 A data file with a sorted index file

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

External Hashing



- In the external hashing scheme, each entry of table contains a pointer to the beginning of a chain, where each chain consists of blocks of index records, i.e., you hash an index file rather than the data file
- Associated with each entry table[i] is a linked chain of blocks of the index file.
- Each block of table[i] 's linked chain contains index records whose keys (and corresponding data records' search keys) hash into location i.
- To form the linked chains, you must reserve space in each block for a block pointer, the integer block number of the next block in the chain

Figure 21-8 A hashed index file

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

External Hashing

Addition when an index file uses external hashing.

1. Add data record into data file.
   a) The new record can go anywhere you want.
   b) If a previous removal has left a free slot in the middle of the data file, you can insert it there.
   c) If no slots are free, you insert the new data record at the end of the last block, or, if necessary, you append a new block to the end of the data file
2. Add corresponding index record into index file.
   a) Insert into the index file an index record that has key value searchKey and pointer value p (number of the block)
   b) Because the index file is hashed, you first apply the hash function to searchKey , letting

      i = h(searchKey)

   c) Insert the index record <searchKey, p > into the chain of blocks that the entry table[i] points to.
   d) You can insert this record into any block in the chain that contains a free slot, or, if necessary, you can allocate a new block and link it to the beginning of the chain

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

External Hashing

Removal when an index file uses external hashing

1. Search index file for corresponding index record
   a. apply the hash function to searchKey , letting i = h(searchKey)
   b. Search the chain of index blocks pointed to by the entry table[i] for an index record whose key value equals searchKey
      i. If you do not find such a record, the data file does not contain a record whose search key equals searchKey.
      ii. If you find an index record <searchKey , p>, you remove it from the index file after noting the block number p , which indicates where in the data fi le you can find the data record to be removed.
2. Remove data record from data file
   a. The data record is in block p of the data file.
   b. Access this block, search the block for the record, remove the record, and write the block back to the file.

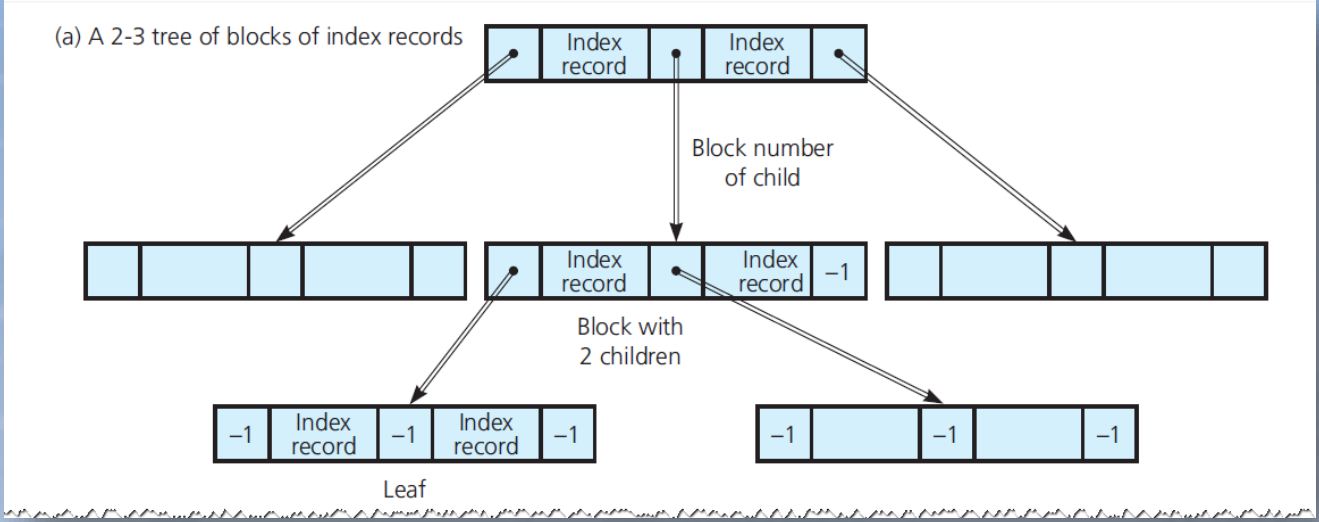
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

External Hashing

- Operations getItem, add, and remove requires low number of block accesses
- Never have to access more than one block of the data file, and at worst you have to access all of the blocks along a single hash chain of the index file
- Take measures to keep the length of each of the chains quite short
- Make the size of the array table large enough so that the average length of a chain is near one block
- Hash function should scatter the keys evenly
- Possibly structure each chain as an external balanced search tree
- The hashing implementation is preferred when you need to perform retrieval, insertion, and removal operations on a large external dictionary.
- This implementation is not practical for certain other operations, such as sorted traversal, retrieval of the smallest or largest item, and range queries that require ordered data.
- When these types of operations are added to the basic dictionary operations, you should use a search-tree implementation instead of hashing

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees



- Each block of the file is a node in the tree and contains three child pointers, each of which is the integer block number of the child.
- A child pointer value of –1 plays the role of a null pointer, and thus, for example, a leaf will contain three child pointers with the value –1.
- If you organized the index file into a 2-3 tree, each node (block of the index file) would contain either one or two index records, each of the form <key, pointer>, and three child pointers.
- The pointer portion of an index record has nothing to do with the tree structure of the index file; pointer indicates the block (in the data file) that contains the data record whose search key equals key.
- To help avoid confusion, the pointers in the tree structure of the index file will be referred to as child pointers.
- You must organize the index records in the tree so that their keys obey the same search-tree ordering property as an internal 2-3 tree.
- This organization allows you to retrieve the data record with a given value in its search key.

Figure 21-10 An index file organized as a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees

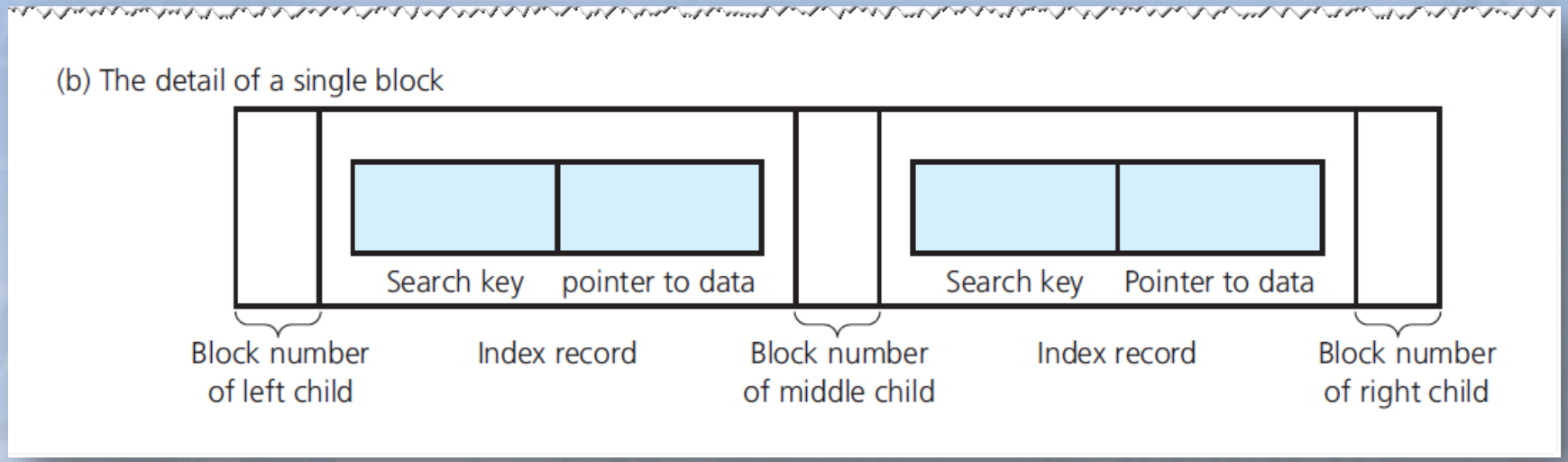


Figure 21-10 An index file organized as a 2-3 tree

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees

Adding a record to a B-tree

1. Add data record to data file
2. Add a corresponding index record to index file
   a. split and merge nodes of the tree just as you do for the internal version

Removing a record from a B-tree

1. Locate index record in index file
2. Remove data record from data file
   a. split and merge nodes of the tree just as you do for the internal version

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees

- Generalize the 2-3 tree to a structure that is even more suitable for an external environment.
- **Add more children per node reduces the height of the search tree** but increases the number of comparisons at each node during the search for a value.
- **In an external environment, the advantage of keeping a search tree short far out weighs the disadvantage of performing extra work at each node**.
- As you traverse the search tree in an external environment, you must perform a block access for each node visited.
- Because the time required to access a block of an external file is, far greater than the time required to process the data in that block once it has been read in, **the overriding concern is to reduce the number of block accesses required**.
- **Reduce the height of the tree**, even at the expense of requiring more comparisons at each node.
- In an external search tree, you should thus **allow each node to have as many children as possible**, with only the block size as a limiting factor.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees

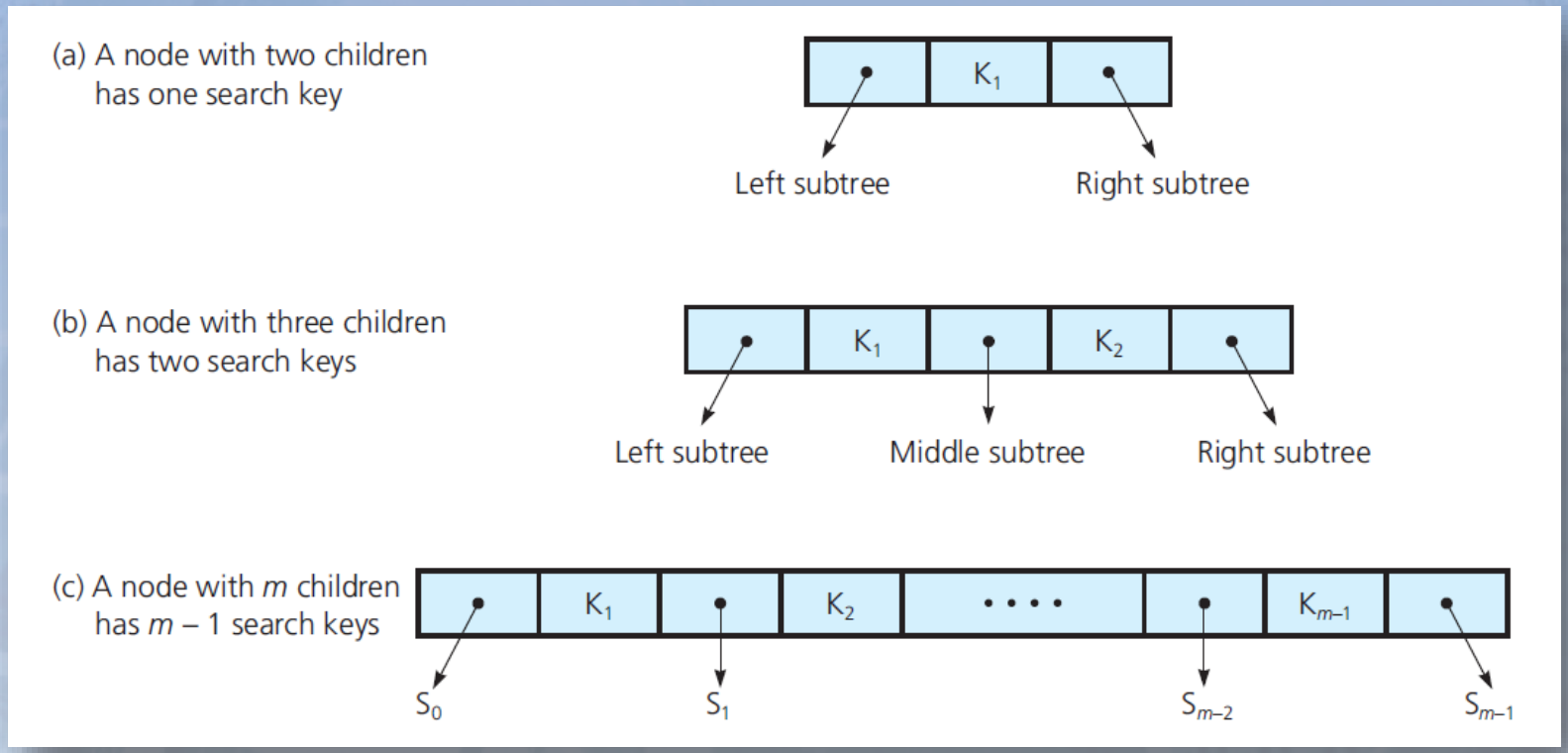
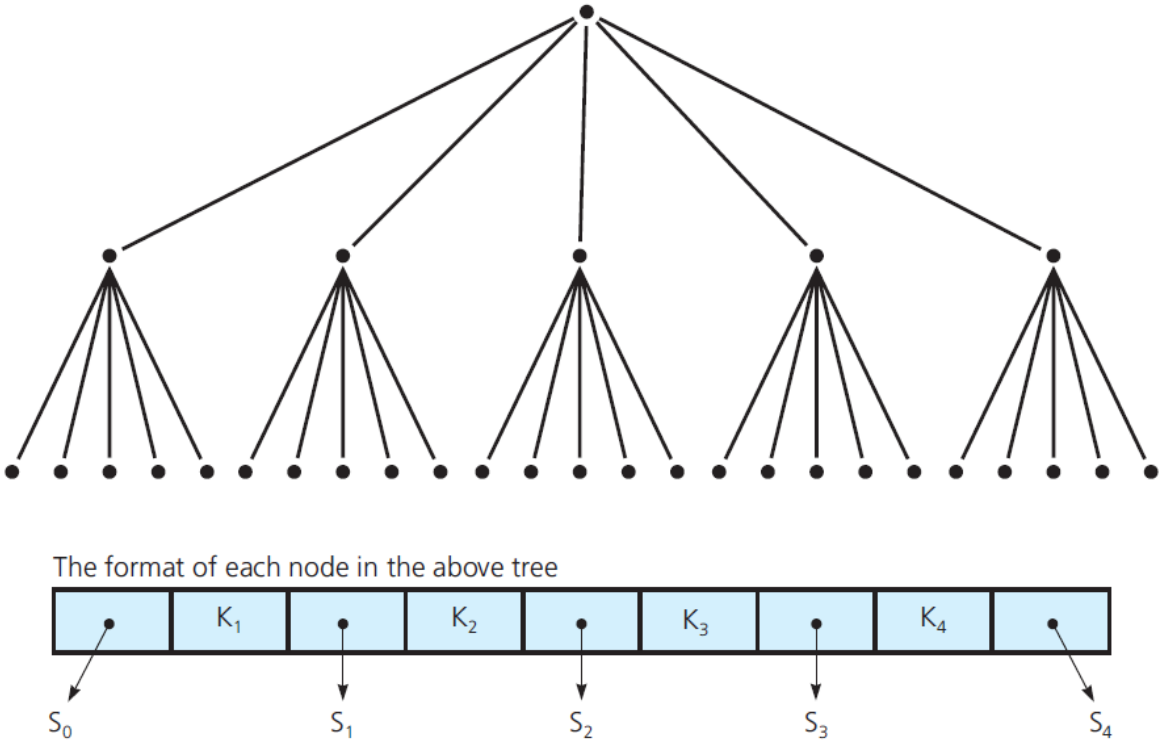


Figure 21-11 Nodes with two, three, and m children and their search keys

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees



This search tree has the minimum possible height, its balance is too difficult to maintain in the face of insertions and removals.

Figure 21-12 A full tree whose internal nodes have five children

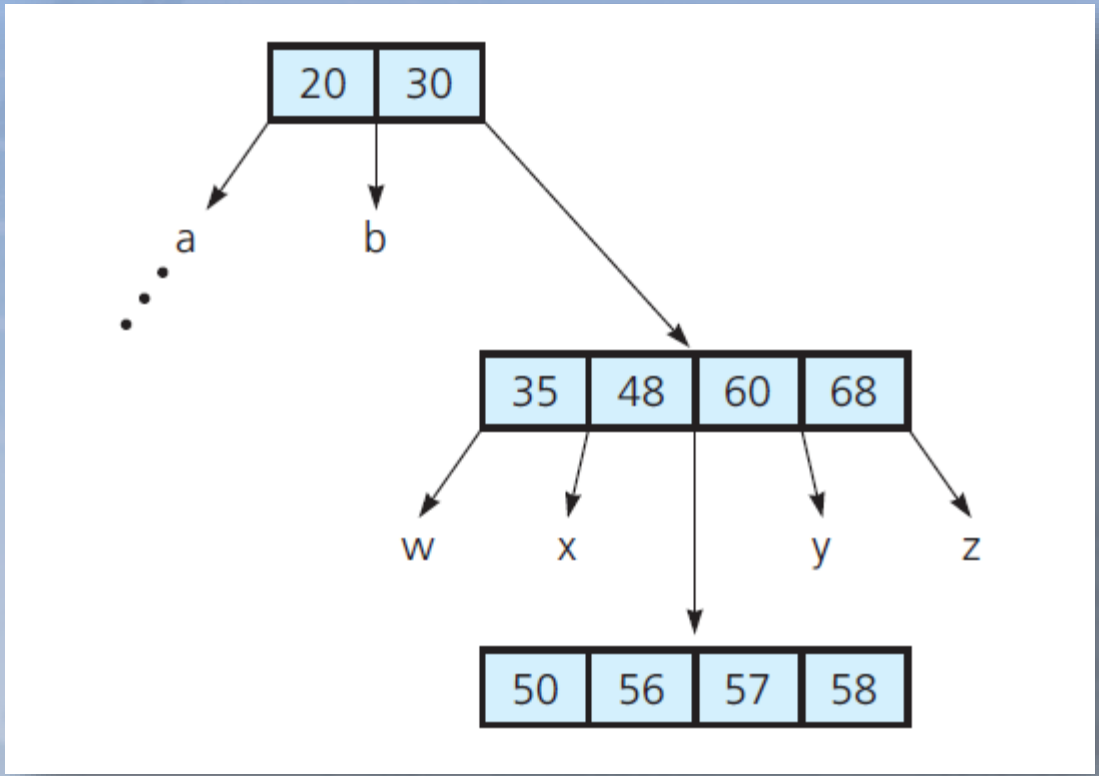
© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees

- B-tree of degree m (number of children nodes) and has the following characteristics:
  - Require all the leaves of the search tree be at the same level—that is, the tree is balanced
  - Allow each internal node to have between m-1 and [m /2] + 1 children, and each internal node has one more child than it has records.
    - An exception to this rule is that the root of the tree can contain as few as one record and can have as few as two children.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees



- 5 is the maximum and 3 is the minimum number of children that an internal node—other than the root—in the tree can have
  - Note: each internal node has between m-1 and [m /2] + 1 children, and each internal node has one more child than it has records (except the root)

Figure 21-13 A B-tree of degree 5

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Multiple Indexing

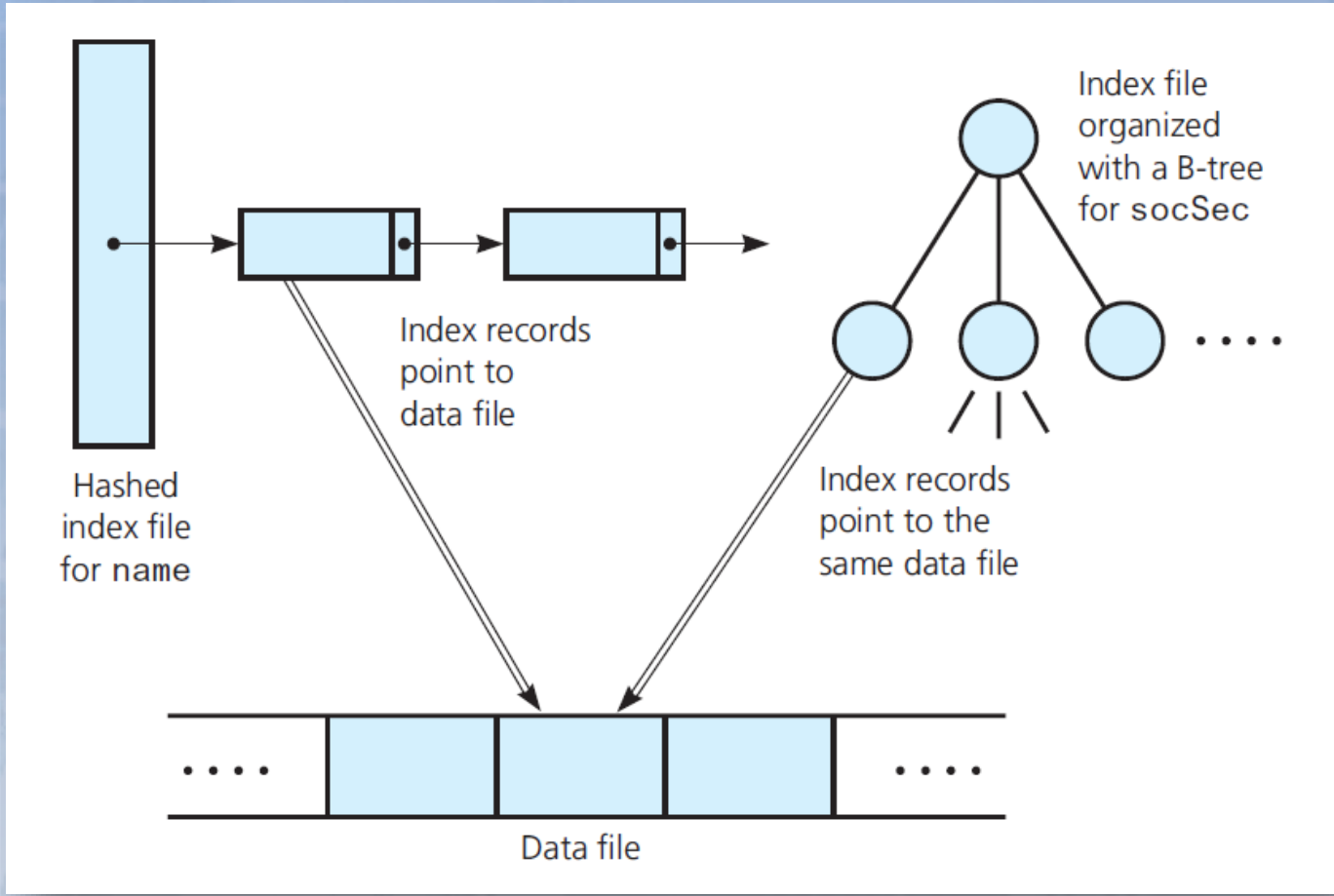


Figure 21-16 Multiple index files

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Summary

- **An external file is partitioned into blocks**. Each block typically contains many data records, and a block is generally the smallest unit of transfer between internal and external memory. That is, **to access a record, you must access the block that contains it**.
- **You can access the ith block of a direct access file without accessing the blocks that precede it.** In this sense direct access files resemble arrays.
- **Before you can process (for example, inspect or update) a record, you must read it from an external file into internal memory**. Once you modify a record, you must write it back to the file.
- Block accesses are typically quite slow when compared to other computer operations. Therefore, you must carefully organize a file so that you can perform tasks by using only a few block accesses. Otherwise, response time can be very poor.
- **You can modify the merge sort algorithm, so that it can sort an external file of records** without requiring all of the records to be in internal memory at one time.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Summary

- **An index to a data file is a file that contains an index record for each record in the data file**. An index record contains both the search key of the corresponding data record and the number of the block in the data file that contains the data record.
- **If a record is inserted into or removed from a data file, you must make the corresponding change to the index file. If a data file has more than one index file, you must update each index file. Thus, multiple indexing has an overhead.**
- **You can organize an index file by using either hashing or a B-tree**. These schemes allow you to perform the basic dictionary operations by using only a few block accesses.
- **Although external hashing generally permits retrievals, insertions, and removals to be performed more quickly than does a B-tree, it does not support such operations as sorted traversals or range queries. This deficiency is one motivation for multiple indexing.**
- **You can have several index files for the same data file. Such multiple indexing allows you to perform different types of operations efficiently, such as retrieval by name and retrieval by Social Security number**.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Introduction to Relational Databases and Indexing

- SQL is a very important language to learn
  - 90% of Software Engineering jobs require SQL
- In the RDBMS
  - Data are stored in tables
  - B-trees are called Indexes

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Efficient Access to Table Data

- Common SELECT looks like this:

SELECT <col list> FROM <table> WHERE <condition>

E.g.

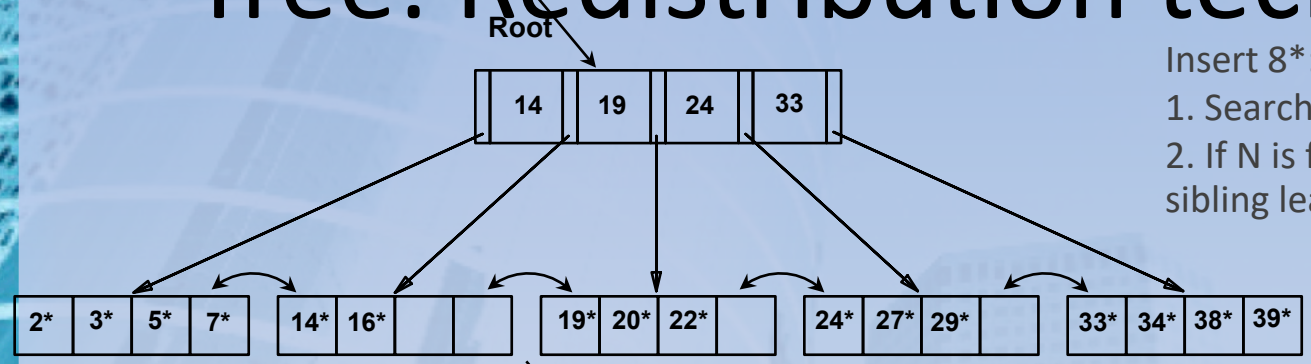
SELECT name, salary FROM Employee WHERE employee_id = 101;

- Simplest approach: Scan every row in the table and check if the condition is met (employee_id = 101)
  - For tables with large number of rows, this will lead to thousands/millions of reads from the disk and will flush the data cache
  - Highly inefficient

Balanced Tree Index

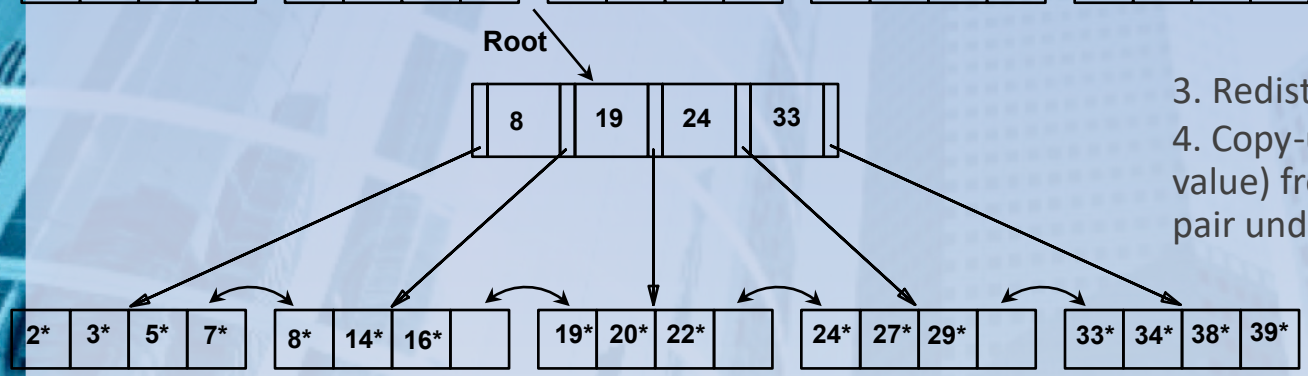
- Not always perfectly balanced
- Need to be close to balanced for O(log n)
- Flexible if large number of entries per node, then don't need constant rebalancing
  - Large key values negate this benefit (i.e., large concatenated column values)
- Sometimes, better to just drop the index and recreate it

Inserting a Data Entry into a B-Tree: Redistribution technique



Insert 8*:

1. Search for the correct leaf node L.
2. If N is full then check the immediate sibling leaf nodes (L-1 and L+1).



3. Redistribute the entries.
4. Copy-up the first entry value (smallest value) from the second node of the node-pair under consideration.

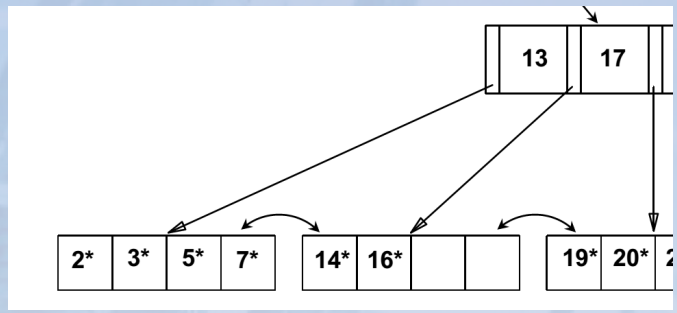
Inserting a Data Entry into a B-Tree: Redistribution technique

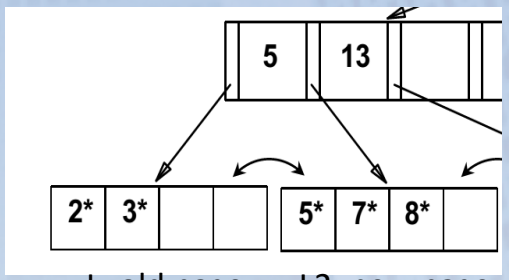
Find correct leaf *L* for the new data entry.

Add the data entry into *L*:

1. If *L* has a free slot, then *done*!
2. Else, must <u>*split*</u> *L (into L and a new leaf node L2)*

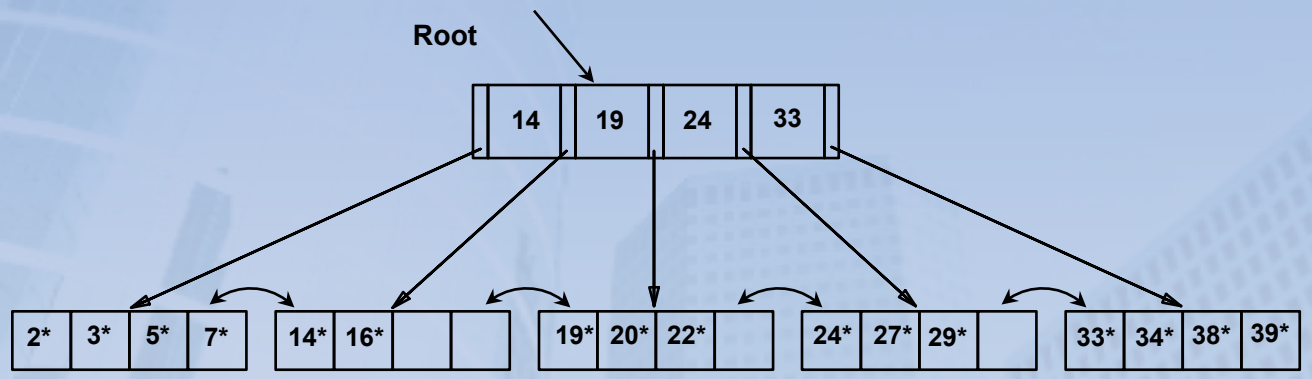
   2A. Redistribute the 2d+1 entries evenly among L and L2





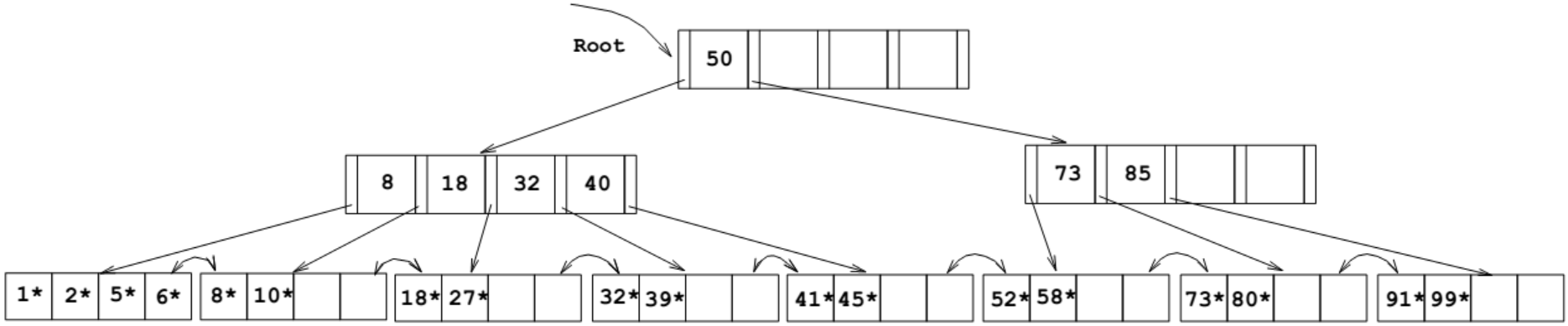
L: old page L2: new page

Inserting a Data Entry into a B-Tree: Redistribution technique

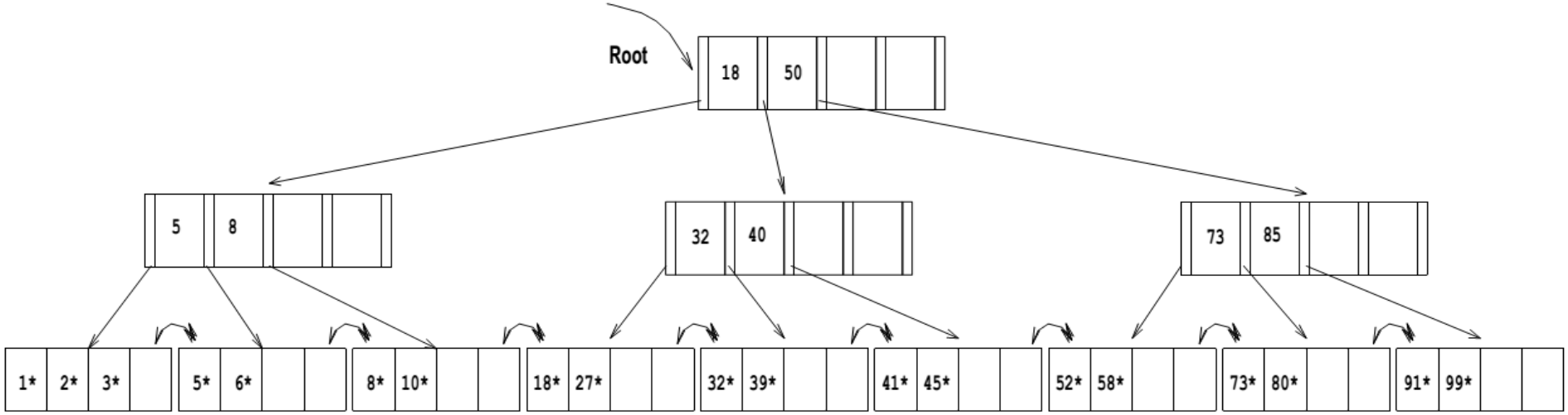


Inserting into a B-Tree

1. If the parent node (P) of the leaf node L has a free entry, then:
   1. **Copy up** the smallest key value in the new leaf node, L2, to the parent node P.
   2. Also, add a pointer to L2 from the node P.
2. Else, must split the node P into nodes P and P2. (These are non-leaf index nodes.)
   1. Sort and compile the 2d+1 entries.
   2. Of the 2d+1 entries: Assign the first d to P, and the last d to P2.
   3. The one middle key entry is **pushed up** to the parent of P, along with the pointer to P2.
   4. Step 2.3 Is applied recursively if the parent node of P is full.
   5. If the root node R is split into nodes R and N then:
      1. Add a new root node R_{NEW}, and add pointers from this new root to nodes R and N, which are both internal nodes now.
      2. Update the pointer for the root to point to R_{NEW}.



- Insert 3*.



© 2017 Anagha Kulkarni

End

Chapter 21

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Chapter 21

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

External Hashing

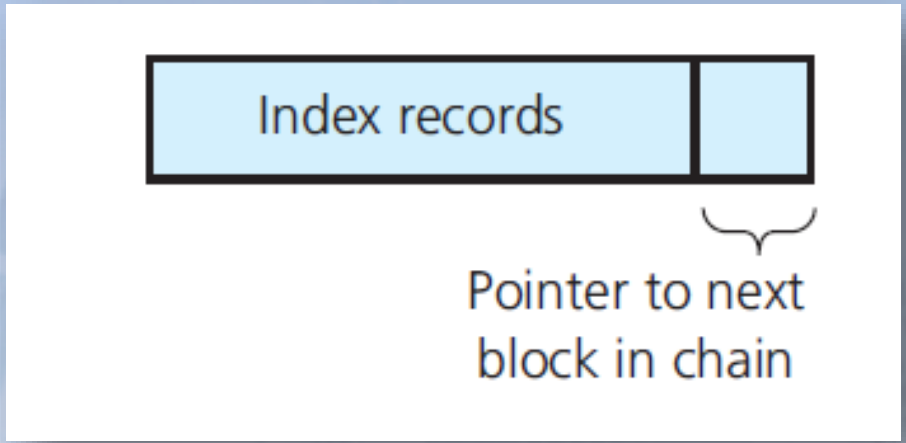


Figure 21-9 A single block with a pointer

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

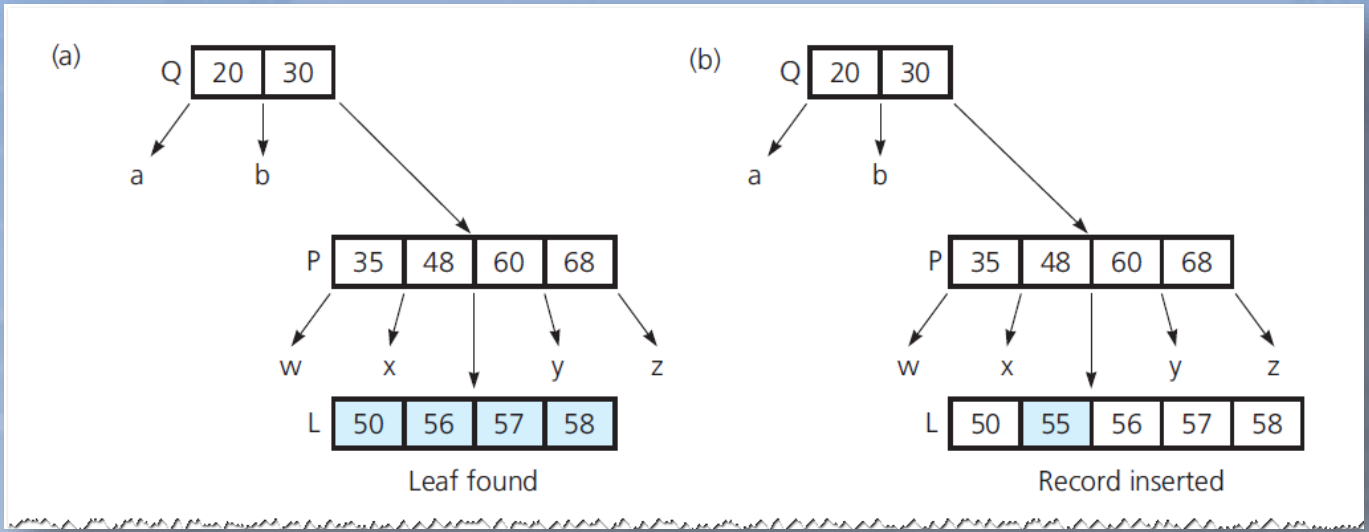
Multiple Indexing

A removal by name must update both indexes

1. Search name index file for jones and remove index record.
2. Remove appropriate data record from data file, noting socSec value ssn of this record.
3. Search socSec index file for ssn and remove this index record.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees



Insertion into a B-tree

Insert the data record into the data file. Find block p in the data file into which you can insert the new record, which is any block with a vacant slot or a new block.

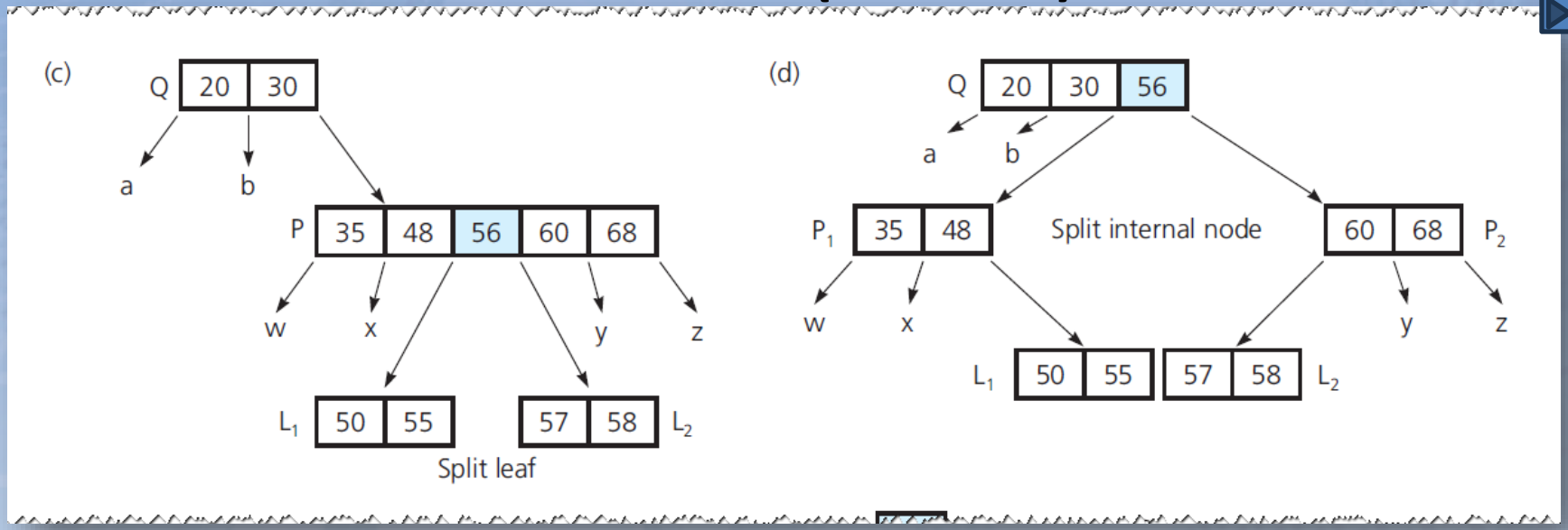
Insert a corresponding index record into the index file.

1. Insert the index record into the index file, which is a B-tree of degree 5. The first step is to locate the leaf of the tree in which this index record belongs by determining where the search for 55 would terminate
2. Insert the new index record into L, causing it to contain five records

Figure 21-14 (a through d) The steps for adding 55 to a B-tree; (e) splitting the root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees (cont.)



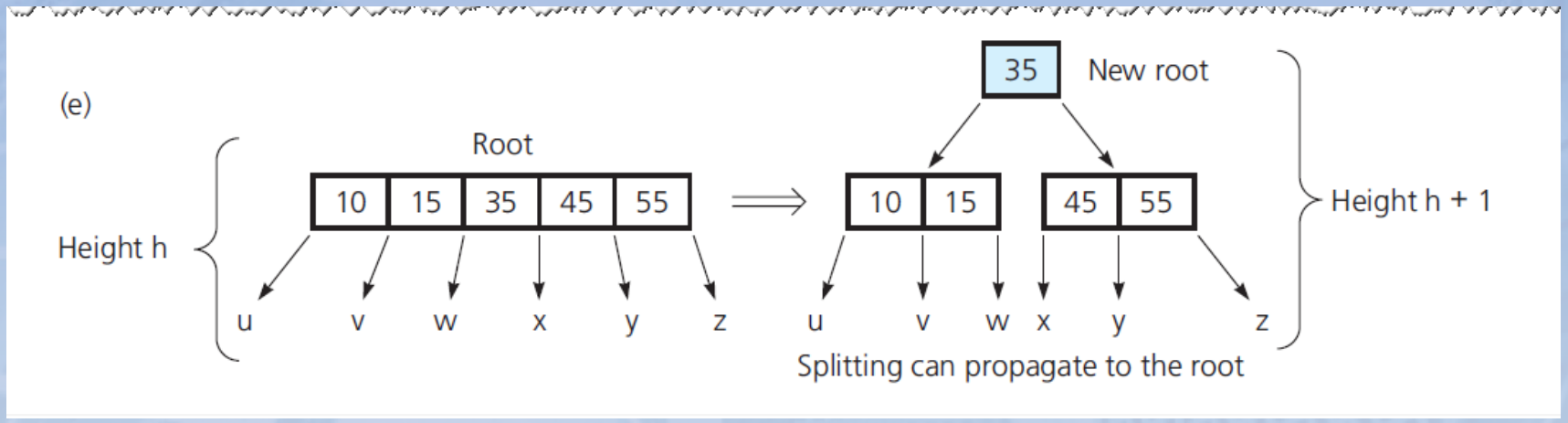
Insert a corresponding index record into the index file (cont.)

3. split L into L1 and L2
4. P now has six children and five records, so it must be split into P1 and P2
5. record with the middle key value (56) is moved up to P ’s parent, Q
6. P ’s children must be distributed appropriately, as happens with a 2-3 tree when an internal node is split (Figure 21-14 d)

Figure 21-14 (a through d) The steps for adding 55 to a B-tree; (e) splitting the root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees

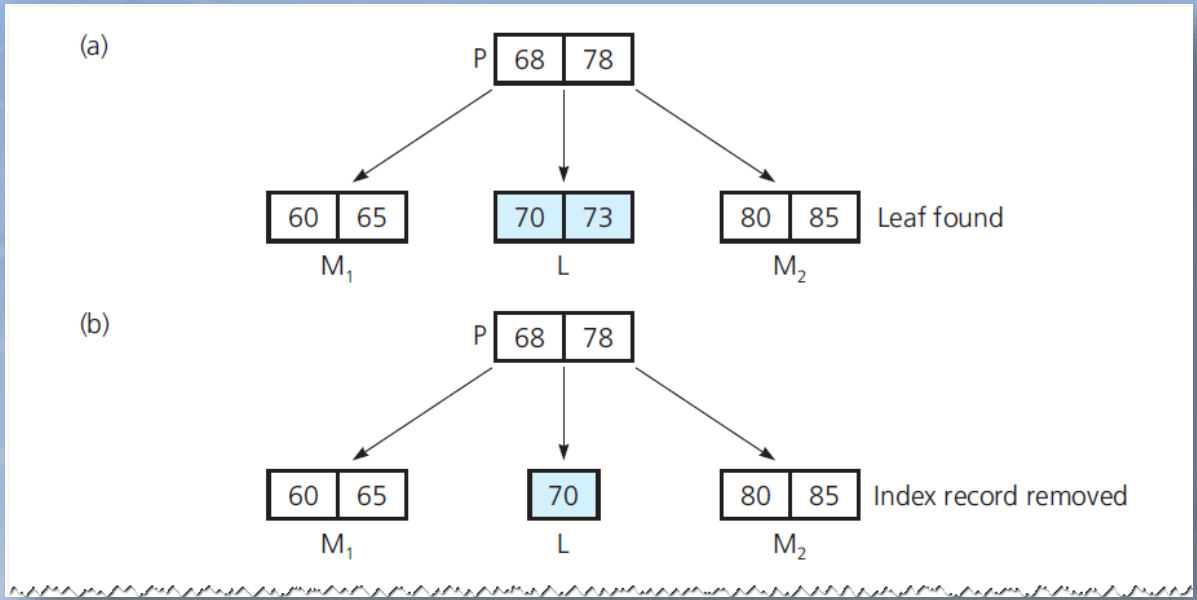


An insertion might cause splitting to propagate all the way up to the root (Figure 21-14 e).

Figure 21-14 (a through d) The steps for adding 55 to a B-tree; (e) splitting the root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees



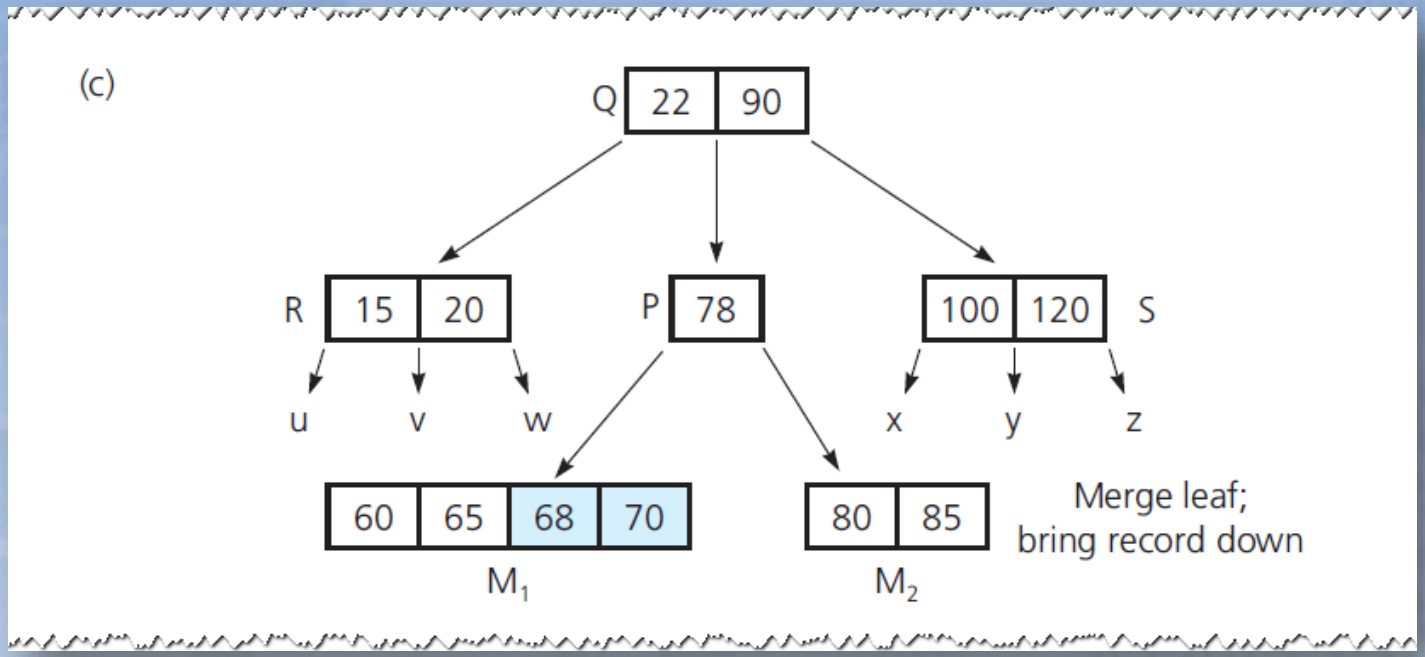
Removal from a B-tree

1. Locate the index record with the desired key value (leaf L).
2. Note value of p (delete data record in subsequent step)

Figure 21-15 (a through e) The steps for removing 73; (f) removing the root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees



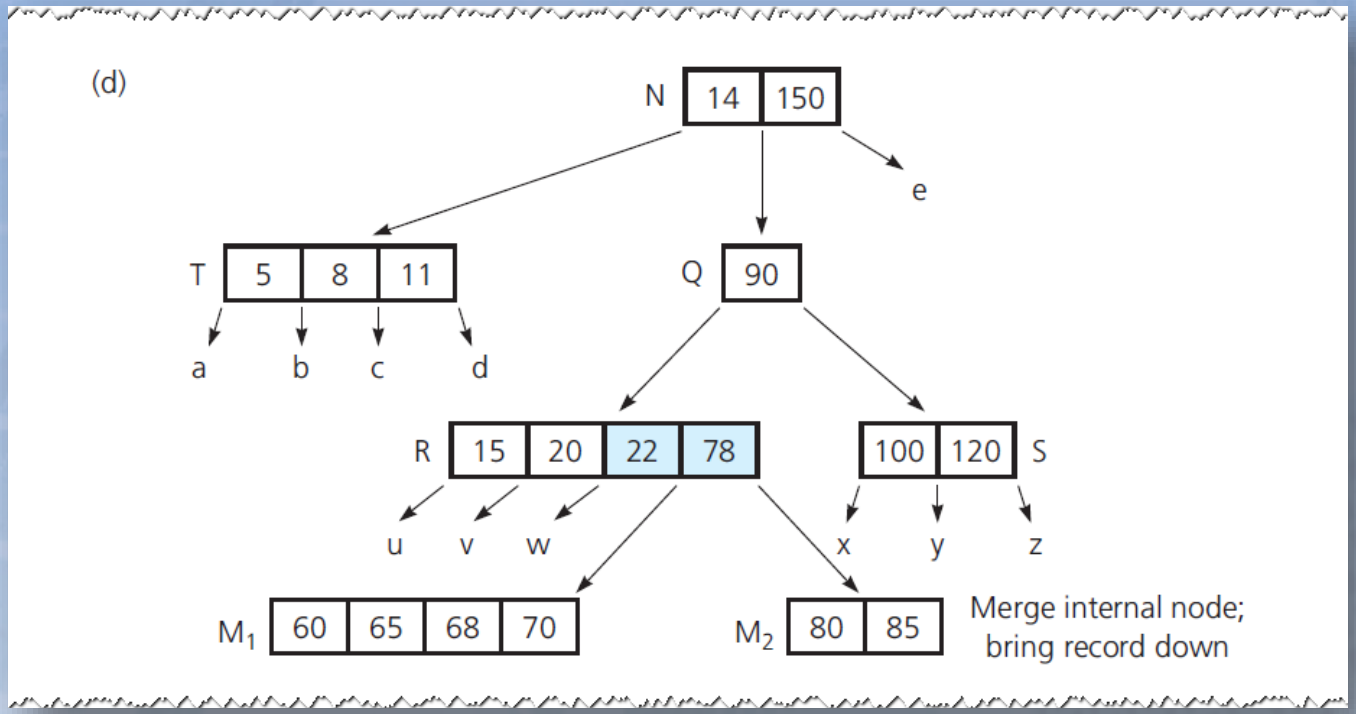
Removal from a B-tree (cont.)

3. L now contains one value (recall that a node must contain at least two values), and since L 's siblings cannot spare a value, you merge L with one of the siblings and bring down a record from the parent P (Figure 21-15 c).
4. P now has only one value and two children, and since its siblings cannot spare a record and child, you must merge P with its sibling R and bring a record down from P's parent, Q . Because P is an internal node, its children must be adopted by R. (See Figure 21-15 d.)

Figure 21-15 (a through e) The steps for removing 73; (f) removing the root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees



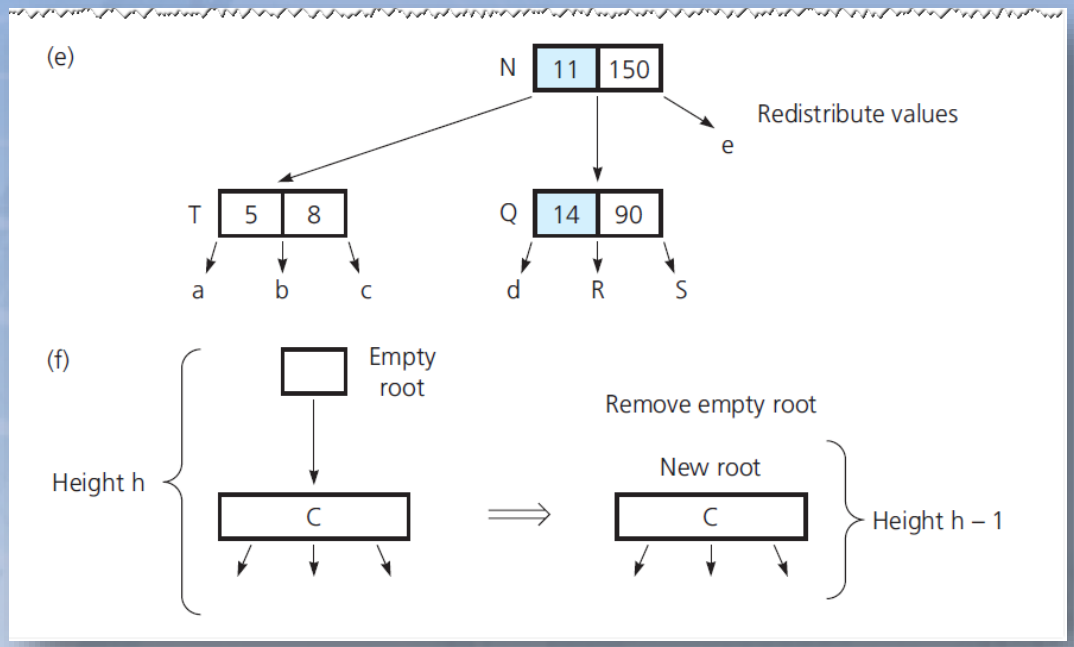
Removal from a B-tree (cont.)

4. After this merge, P's parent Q is left with only two children and one record. In this case, however, Q 's sibling T can spare a record and a child, so you redistribute children and records among T , Q and the parent N to complete the removal. (See Figure 21-15 e.)

Figure 21-15 (a through e) The steps for removing 73; (f) removing the root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

B-Trees



Removal from a B-tree (cont.)

5. If a removal propagates to the root, leaving it with one record and two children, you are finished because the definition of a B-tree allows this situation.
6. If a future removal causes the root to have a single child and no records, you remove the root so that the tree's height decreases by 1, as Figure 21-15 f illustrates.
7. The removal of the index record is complete, and you now must remove the data record, by accessing block p (noted from initial step), delete data record, and write to file.

Figure 21-15 (a through e) The steps for removing 73; (f) removing the root

© 2017 Pearson Education, Hoboken, NJ. All rights reserved